

Digital Brain

Digital Brain turns the documents, tickets, incidents and code a team already owns into **evidence the business can stand behind** — then puts that evidence to work in the two places it pays back fastest: **software delivery** and **service operations**.

Overview

Why we are building it

THE PROBLEM

The knowledge that keeps delivery and operations moving is real, but it is scattered — a spec here, a Jira thread there, a postmortem nobody re-read, and the code itself. Finding it is slow, and it walks out of the door with the people who hold it.

WHY GENERAL AI DOES NOT SOLVE IT

Off-the-shelf assistants answer from training data, so a confident wrong answer looks exactly like a right one. There is no way to check the working — which is why they stay out of anything that carries real risk.

WHAT WE ARE BUILDING INSTEAD

Digital Brain keeps our own material as immutable, digested sources and answers **only** from them — quotes attached, re-checked against the live source at the moment of answering. If it cannot be evidenced, it is not shown.



One evidence base, two consumers. Everything later in this page is a detail of this picture.

● In the software lifecycle

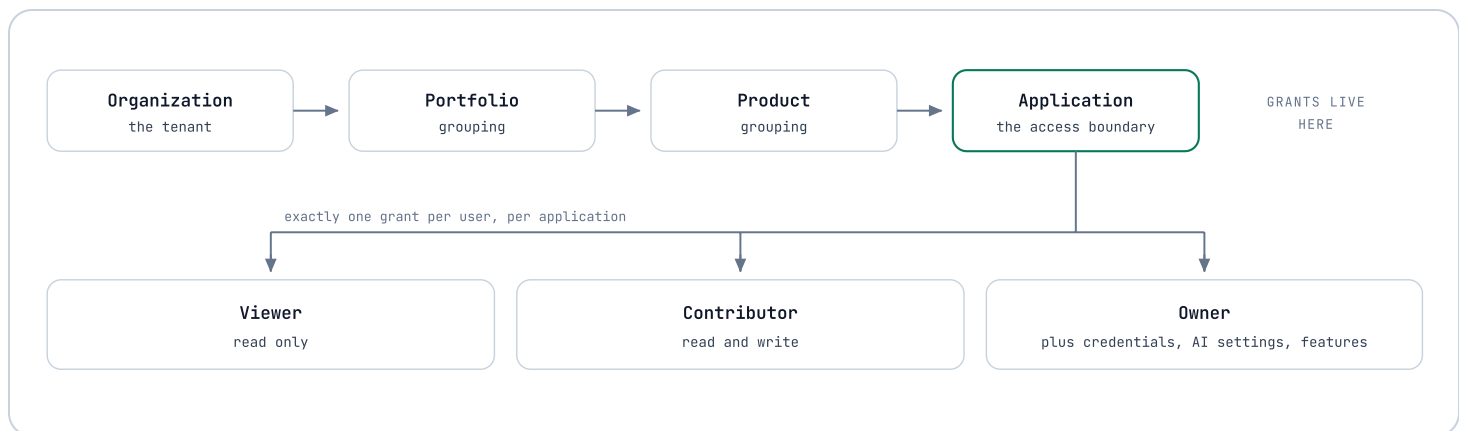
- **Onboarding** — a new engineer asks the system instead of interrupting a senior one, and gets the source, not a summary
- **Analysis** — a ticket is checked against what we have actually written down, so gaps surface before work starts
- **Change** — a bug ticket becomes a draft pull request with a stated rationale; engineers review a proposal instead of a blank page
- **Traceability** — knowledge and a pinned code snapshot sit side by side, so "why is it like this" has an answer with a date on it

● In service operations

- **Triage** — "have we seen this before" is answered from our own incidents and runbooks, with quotes, in seconds
- **Escalation** — the responder gets evidence rather than a guess, and can see exactly which document it came from
- **Safety** — the platform reads; it never executes a runbook, updates a ticket or touches the live service
- **Learning** — the postmortem is imported like any other document, so the next incident starts better informed

Both sides read the **same approved evidence**, inside one boundary: the application. A team's sources, knowledge, credentials and API keys are scoped to it and nothing crosses — which is what makes one platform safe to share across business units.

Organization, applications and roles



Neither admin role carries application access. A platform administrator and an organization administrator both get **404** — not **403** — on an application they hold no grant for, and creating an application does not grant one. **Approval is a separate flag**, not a fourth role: it can sit on any grant, and the approver can never be the person who wrote the plan.

01 Technology & features

Server-rendered Django, **no JavaScript build step**. Four layers, and one row per thing we use.

Browser	server-rendered HTML · chat.js · modal.js · no build step
Application	Django 5.2 · Waitress · access policy · audit · worker
Platform core	knowledge graph · retrieval · Code Factory · connectors · MCP
Data & AI	PostgreSQL · secret files 0600 · Claude Agent SDK · OpenAI Agents SDK

LAYER	TECHNOLOGY	ROLE
Frontend	Server-rendered Django templates, <code>chat.js</code> , <code>modal.js</code>	Whole interface; no build step, no bundler. Every control still works with JavaScript off
Backend	Python 3.12 + Django 5.2 + Waitress	Request handling, access policy, audit ledger, background worker
Data	PostgreSQL + psycopg 3 · SQLite locally	Sources, graph revisions, usage receipts, audit events
Ingestion	Markdown (Microsoft)	Any document → clean Markdown; one library, all formats. Runs offline, no network
Connectors	GitHub · Jira (JQL) · ServiceNow · SharePoint via MS Graph	Bounded manual pulls; each record becomes an ordinary immutable source
Knowledge graph structure	Deterministic build, pure Python — no model	Markdown tables and headings → graph with a quote on every edge. Free, rebuilt automatically whenever a source changes
Knowledge graph enrichment	Claude or OpenAI, on request only	Adds relationships the structure cannot express. Every one must carry a quote that is re-verified against the source. The only paid graph step
Retrieval	Term ranking + one-hop expansion	Returns a subgraph and its source citations; reproducible, no embeddings
Code graph	git clone + Python <code>ast</code>	A repository at one commit → files, imports, definitions, roles
AI	Claude via Claude Agent SDK · OpenAI via Agents SDK	Answer generation and the Code Factory phases; a separate credential per application, receipted per turn
Machine API	REST <code>/api/v1/</code> · MCP server <code>brain</code>	Same verified evidence for tools and agents; calls no model
Secrets	Files on disk, mode 0600	One per provider per application; never a database row, never an environment variable
Security	Argon2 · django-axes · strict CSP · defusedxml · Pillow	Password hashing, login lockout, no third-party script, hardened parsing and image handling

Where AI is used

Most of the platform runs without a model. Retrieval is always deterministic; a model only ever writes over evidence that was already found.

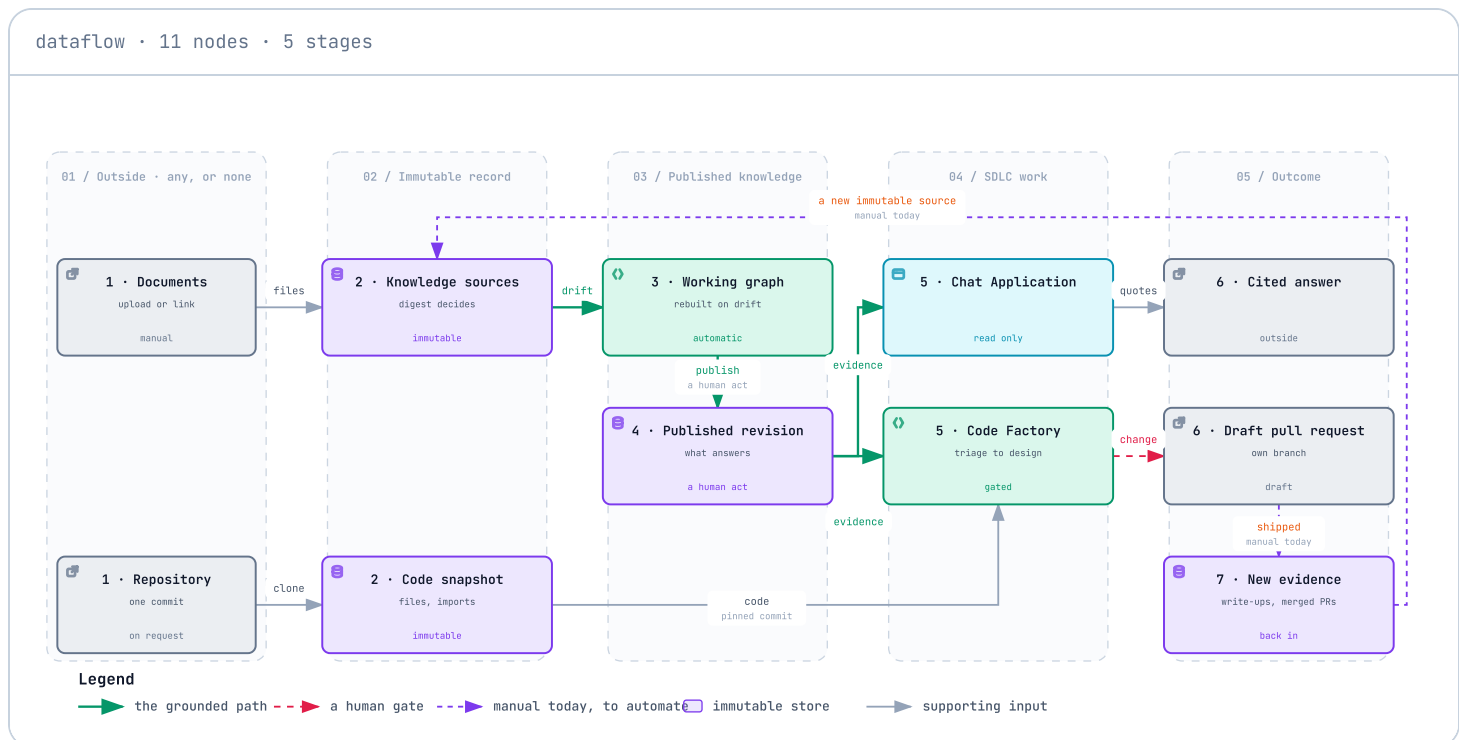
SURFACE	WITHOUT A MODEL	MODEL CALL
Knowledge graph	Structural build from tables and headings — automatic, free, runs on every source change	Enrichment only , and only when someone requests it. Relationships must carry a quote that is re-verified
Chat	Sources mode returns the matching evidence directly — no model at all. Retrieval is deterministic in every mode	AI and Graph answer modes: one call to write the answer over evidence already retrieved
Code Factory	Verification — path safety, elisions, and a staleness re-check of every file. Delivery — branch and draft PR	Triage · analysis · design · implementation — four phases, each with its own token budget
REST & MCP	Everything. Same verified evidence, served straight from the published graph	Never — a machine caller cannot spend the AI budget

Features are switched on **per application** — each team gets only what it asked for. Two ship restricted:

Chat API — opt-in (the only machine surface that spends the AI budget) and **Document uploads — dev only** (until scanning is signed off for production).

02 System overview

The whole platform in seven steps. A team starts with whatever it has — documents, a repository, or both; neither is required. Everything that arrives is kept immutable, the knowledge is refreshed automatically at no cost, and **a person decides when it becomes the version that answers**. Both ways of working then read that same approved version, and what they produce comes back in as new evidence.



● Steps 1-4 · building the evidence

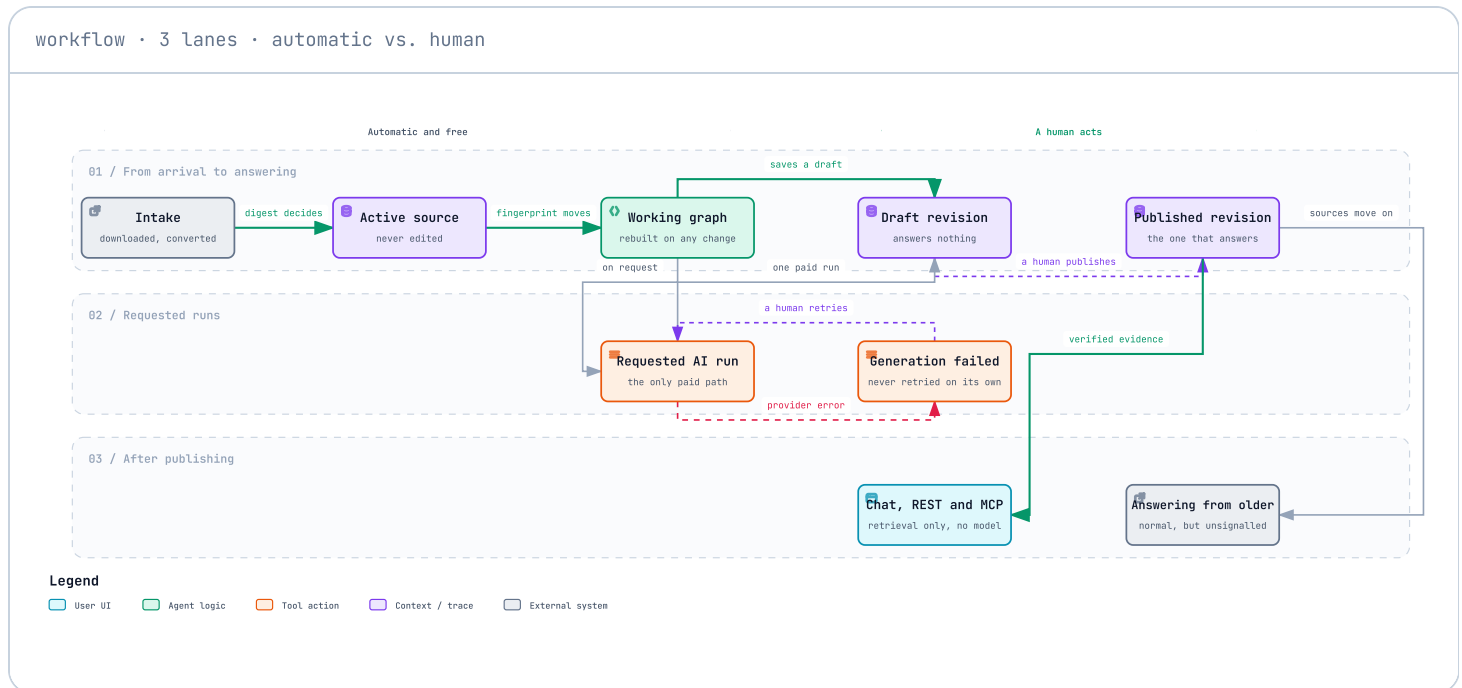
- Documents or a repository — neither one is required
- Each becomes an immutable source; code stays separate
- The worker rebuilds the graph whenever a digest changes
- A person publishes a revision; nothing answers until then

● Steps 5-7 · using it, closing the loop

- Chat and Code Factory read that same published revision
- A cited answer, or a draft pull request behind two gates
- What returns: a postmortem, an RFC, a merged pull request
- Dashed edges are carried by a person today — proposed to be automatic

03 Knowledge graph management

How the knowledge stays current without running up a bill. Keeping it in step with the sources is automatic and **costs nothing**; the paid AI step happens only when someone asks for it, so a routine refresh can never repeat a charge. And nothing new answers a question until a person approves it.



● Currency is automatic; publishing is not

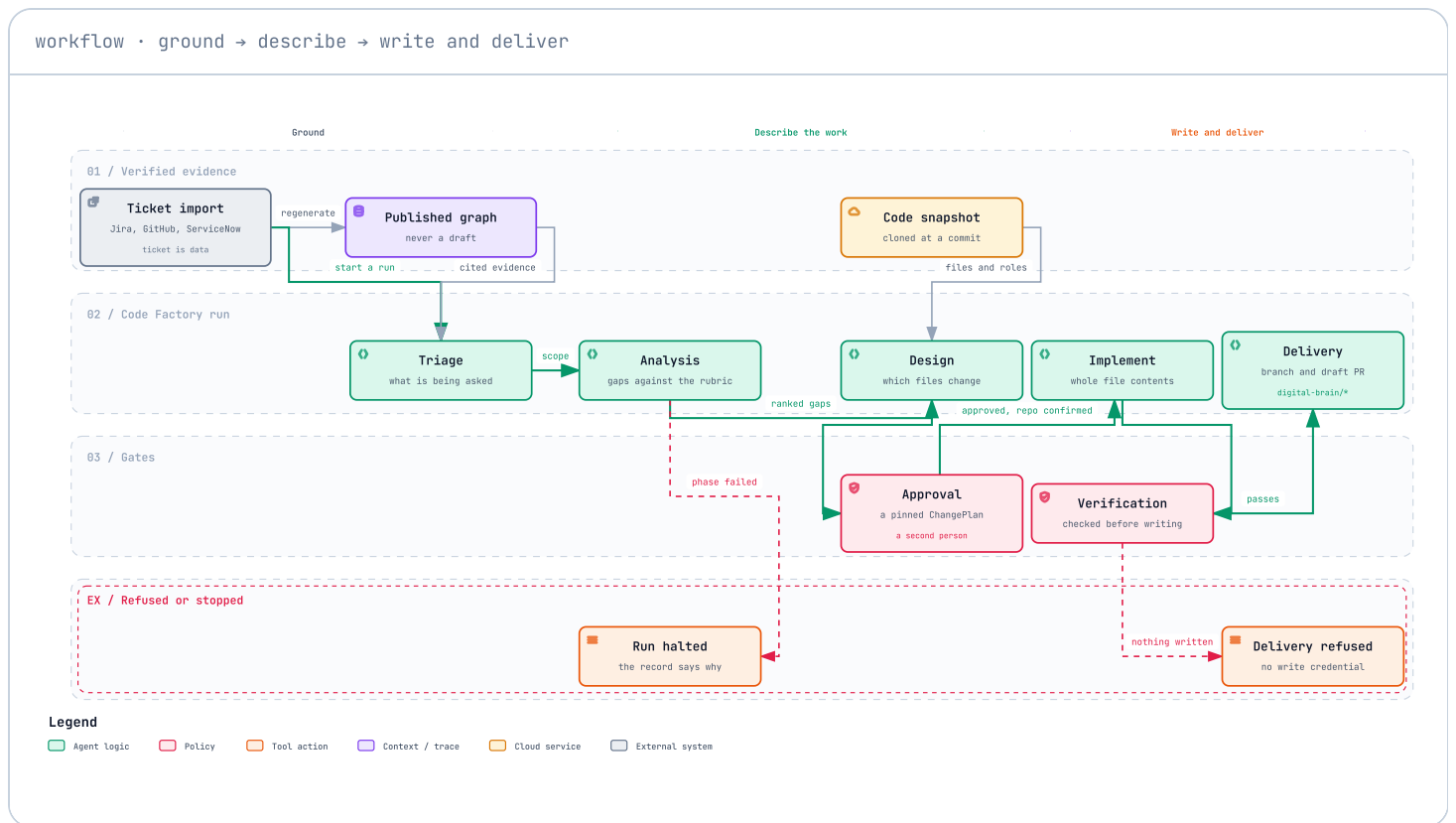
- The worker compares a fingerprint over every active source and rebuilds on any change
- A structural rebuild is free and always saves a draft revision
- Only a human promotes a draft, and publish refuses one whose evidence has moved
- A failed generation is never retried on its own — a person asks again

● Who reads it, and how

- Chat, `/graph/search/` and the MCP server all answer from the published revision
- Neither machine surface calls a model, so no caller can spend the AI budget
- A request may pin a version; every response states which one answered
- No published revision is a `409` — never a quiet fallback to a draft

04 SDLC · bug fix

From a ticket to a reviewed draft pull request. The system works out what is being asked, what is missing and which files would change — then **stops and shows its work**. Approving that plan is one decision by a second person; writing to the repository is a separate decision with its own credential. Engineers stay the reviewers, never the bystanders.

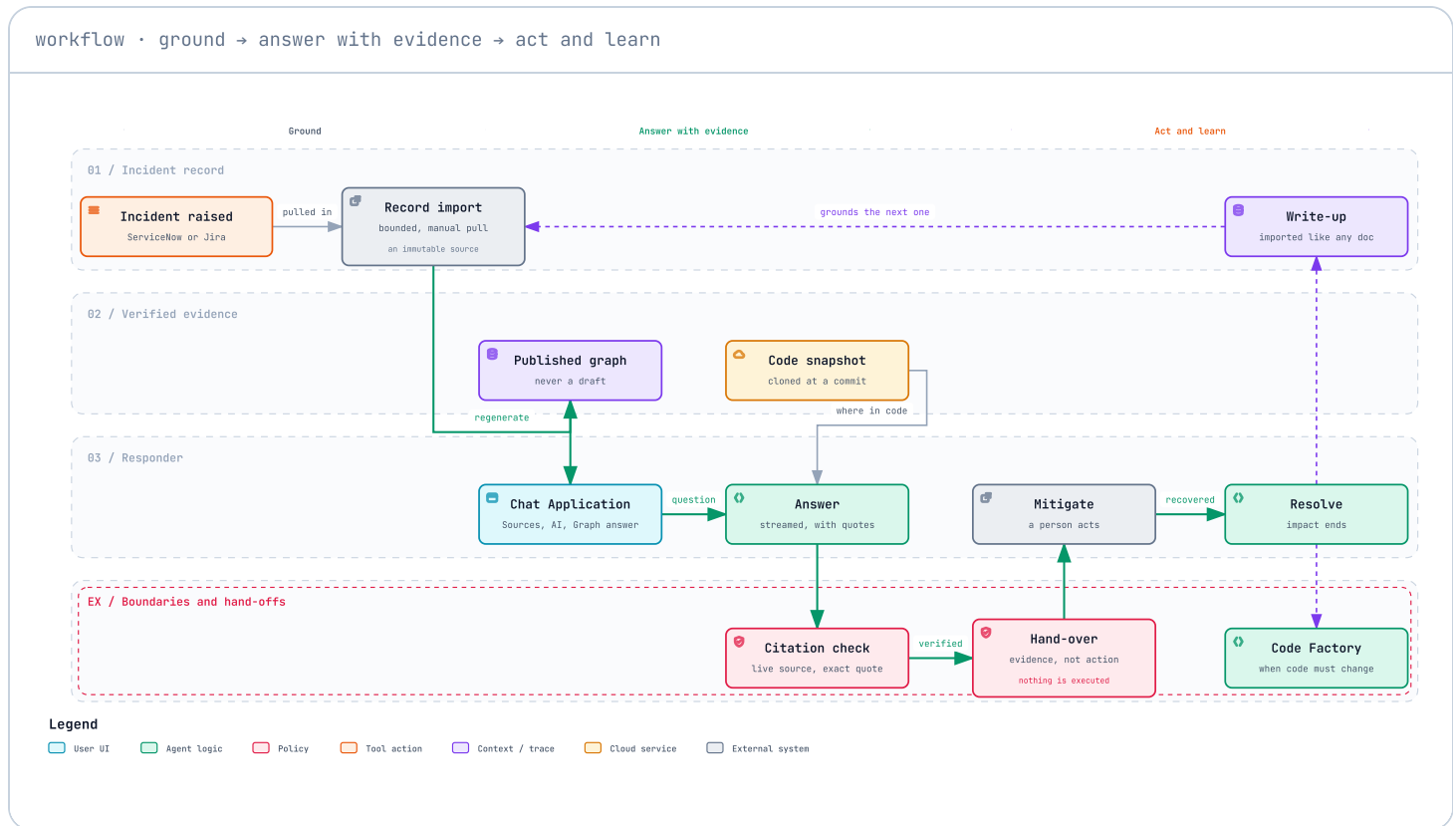


- **The ticket is data, never instruction**
 - Ticket text is evidence about a want, not a command to obey
 - A repository named in a ticket is confirmed by a person first
 - Each cited quote is re-checked against the live source
 - A failed phase halts the run and the record says why

- Two gates, both owned by people
- Approval needs a *different* user holding the approve permission
- Approving a description does not authorise a repository write
- Verification runs before anything is written; failure means nothing is written
- Delivery opens a draft PR on its own `digital-brain/*` branch

05 SDLC • incident management

During an incident the platform gives the responder verified evidence — and **touches nothing**. No runbook is executed, no ticket is updated, no action is taken on the live service. The human stays in command. Afterwards the write-up is imported like any other document, so the next incident starts better informed than the last.



● An incident record is just a source

- ServiceNow, Jira and GitHub imports are bounded and manual, not a mirror
- Each record becomes an immutable source with a SHA-256 digest
- A changed record archives the old revision instead of overwriting it

● The platform never touches the service

- No runbook is executed, no ticket is updated, no action is taken
- REST and MCP callers get the same verified evidence and call no model
- A code change goes back through the Code Factory gates, not around them

A Production posture

What is built and demonstrable today, and exactly what go-live still needs. Both are stated, deliberately — nothing below is claimed that cannot be shown.

- **In place today**

- ✓ Per-application access, re-checked on every request; revocation takes effect immediately
- ✓ CSRF, secure cookies, HSTS, TLS redirect, strict CSP, Argon2, database-backed login lockout
- ✓ Health endpoints and JSON request logs carrying a request ID
- ✓ Every mutation and its audit event share one database transaction
- ✓ Usage receipted per model turn, per provider request ID
- ✓ The full Claude path proven end to end against a live model

What go-live needs

REQUIREMENT	WHY IT BLOCKS GO-LIVE	WHAT IT TAKES
Enterprise SSO & MFA	Identity must come from the corporate directory, not local accounts	Identity provider integration; a separate audience for platform admins
Database-level isolation	Tenant separation currently rests on tested application policy, not on the database	PostgreSQL row-level security and restrictive runtime roles
Document scanning	Uploads stay disabled in production until files are scanned and converted in a sandbox	The hook is built — it needs a scanner installed, configured and kept current
Storage identity	Documents need per-application storage and source-level access control	Per-application storage identities and ACL enforcement
Monitoring & alerting	No availability target can be committed to without measurements behind it	External uptime checks, alert destinations, centralised traces and metrics
Spend controls	Nothing currently caps what one application can spend with a provider	Per-tenant quotas, token and cost budgets, worker concurrency limits
Audit integrity	A database operator can still change audit records	Retention policy, export, and tamper evidence
Recovery	Backups are a plan, not a proven capability, until a restore has been run	Managed backups with point-in-time recovery; tested restore and isolation drills
CI coverage	AI credential isolation is verified by a live run and code review, not on every commit	A funded provider test account so CI can exercise the real path
Accessibility	No conformance can be claimed on a first visual pass alone	Keyboard and screen-reader testing, formal WCAG contrast checks

Nothing ships without clearing four gates

```
ruff check digitalbrain platform_core scripts manage.py
manage.py makemigrations --check --dry-run
manage.py test
scripts/production_preflight.py
```

B Security

These are **invariants, not settings** — properties a future change cannot quietly erode, because each one is pinned by an automated test that fails the build. Weakening any of them is a recorded decision with a written trade-off, which is exactly what an assessor will ask to see.

● Deny by default

Application access is re-evaluated per request against current database state. Platform administration does not imply application access, and revoking a grant takes effect on the next request — including for API tokens.

● The AI cannot widen its own reach

Agent tools are locked to one application and one user *before* a run starts. The model can narrow what it looks at; it has no way to ask for more. A prompt-injection attempt in a ticket or document therefore has nothing to escalate to.

● Citations verified against live sources

Before anything is stored or rendered: source still active, digest unchanged, excerpt still present. What the model claims is not evidence.

● Credentials never enter the database

One file per provider per application, readable only by the service account. Never an environment variable, never a database row, never written to an audit log, never sent back to a browser. A database backup therefore contains no usable secret — only who set it, when, and a digest.

● The server cannot be turned into a probe

Exactly one place fetches a user-supplied URL. It refuses private, loopback and reserved addresses, then pins the connection to the address it validated so DNS cannot change the answer underneath it. http/https only, no redirects, 8 MB and 20 s caps, no credentials attached — the standard server-side request forgery path is closed.

● Revoking access closes the API too

An API token resolves to a person and then runs the same checks their browser session runs. There is no separate token permission model to keep in sync, so withdrawing someone's access shuts their integrations off with no extra step. Token secrets are stored hashed and compared in constant time.

● Document conversion has no network

Files are converted in a subprocess that cannot open a socket at all. A malicious document has nothing to call home to.

● No AI spend goes unrecorded

A missing or unreadable token count is treated as an error, not a free request — because the provider will still bill for it. Every model turn produces its own receipt against its own provider request ID.

- **No shared AI account**

Each application uses its own provider credential, and an AI run is started with a blanked environment so it cannot inherit another application's key or an operator's personal login. This is what makes per-team AI billing and attribution real rather than nominal. Production refuses the development shortcut outright.

- **Model output cannot script the page**

What a model writes is always treated as text, never as markup, and it is escaped on the server before it reaches a browser — deliberately producing no clickable links. A model talked into emitting a malicious payload has no way to execute it in a user's session.